Sequence Alignment – Measured Results

This document records the actual output of running `compare.py` and `benchmark.py` on this machine, as a concrete companion to the algorithm descriptions in `project_description.tex`.

1 Correctness Check (`compare.py`)

200 random trials, each comparing the basic DP alignment against Hirschberg's algorithm:

Metric	Result
Identical optimal cost	200 / 200 (100%)
Byte-identical alignment pair	48 / 200 (24%)

PASS – Hirschberg matches the DP optimum on every trial. The two algorithms agree on cost in every case; they produce different alignment strings 76% of the time, which is expected whenever multiple alignments tie at the optimal cost.

2 Benchmark (`benchmark.py`)

Measured on this machine, sweeping $m+n \in \{128, 256, 512, 1024, 2048, 3072, 4096\}$. Time is wall-clock CPU time; memory is peak `tracemalloc` allocation.

$m+n$	Basic (ms)	Efficient (ms)	Basic peak (KB)	Efficient peak (KB)	Mem. saved	Time cost
128	23.9	28.0	164.8	9.3	94.3%	1.17×
256	95.5	174.7	652.8	19.9	96.9%	1.83×
512	389.4	482.2	2,594.5	40.8	98.4%	1.24×
1024	3,252.5	3,132.9	10,318.7	82.3	99.2%	0.96×
2048	14,854.6	15,100.1	41,128.9	165.0	99.6%	1.02×
3072	37,093.6	39,890.1	92,416.5	247.7	99.7%	1.08×
4096	70,814.1	63,848.1	164,184.6	330.0	**99.8%**	0.90×

Memory. The memory trend is clean and matches theory exactly: the basic table grows quadratically while Hirschberg's rows grow linearly, so the reduction climbs monotonically from 94.3% at $m+n = 128$ to **99.8%** at $m+n = 4096$ (164.2 MB down to 0.3 MB).

Time. The time ratio is noisier on this run than the theoretical $O(mn)$ constant-factor overhead would suggest – Hirschberg's is up to 1.83× slower at small sizes, but actually comes out *faster* than the basic DP at $m+n = 1024$, 2048, and 4096 (ratios below 1.0×). Both algorithms are still $O(mn)$ asymptotically; the inversion is measurement noise from other load on this machine during the run (e.g. background processes, CPU frequency scaling, cache effects) rather than a change in algorithmic behavior. The overall time cost at the largest size measured previously (see `project_description.tex`) was consistently above 1.0×; re-running this benchmark on a quieter machine would be expected to reproduce that pattern more cleanly.

Takeaway. The memory result is the robust, reproducible finding of this project: Hirschberg's algorithm cuts peak memory by over 99% at larger input sizes, confirming the $O(mn)$ vs. $O(m+n)$ space trade-off. The time overhead is real but small and system-noise-sensitive at the sizes tested here.